```
#include <SPI.h>
#include <LoRa.h>
#include <ESP8266WiFi.h>
#include <WebSocketsServer.h>
#include <LittleFS.h>

#define DEBUG 1
#define debug 1

// WIFI
#define WIFI_SSID     "BaseStation"
#define WIFI_PASSWORD "12345678"

// WEBSOCKET
#define WEBSOCKET_PORT 81
WebSocketsServer webSocket = WebSocketsServer(WEBSOCKET_PORT);

// OFFLINE EVENT STORAGE
#define EVENTS_FILE "/events.txt"
#define EVENTS_TEMP_FILE "/events_tmp.txt"

unsigned long nextEventID = 1;

// LoRa Pins - ESP8266
#define LORA_SS     15   // D8
#define LORA_RST    16   // D0
#define LORA_DIO0    4   // D2

// LoRa
#define LORA_FREQUENCY 433E6

// LORA COMMAND ACK / RETRY
#define COMMAND_RETRY_INTERVAL 5000UL
#define COMMAND_MAX_RETRIES    3
#define COMMAND_QUEUE_SIZE 10

String commandQueue[COMMAND_QUEUE_SIZE];
byte commandQueueHead = 0;
byte commandQueueTail = 0;
byte commandQueueCount = 0;

bool commandPending = false;
String pendingCommand = "";
String pendingAnimalID = "";
```

```
unsigned long lastCommandSend = 0;
byte commandRetryCount = 0;

// BASE STATION BATTERY
const int BATTERY_PIN = A0;
const float R1 = 10000.0;
const float R2 = 10000.0;

// MULTI-NODE POLLING
#define MAX_ANIMAL_NODES 10
String animalIDs[MAX_ANIMAL_NODES] =
{"A01","A02","A03","A04","A05","A06","A07","A08","A09","A10"};
bool animalLive[MAX_ANIMAL_NODES] = {false};
byte pollIndex = 0;
int pollAnimalIndex = -1;
bool pollPending = false;
unsigned long pollSendTime = 0;

#define POLL_TIMEOUT 30000UL  // SF12 needs a reasonably long response window.
//#define POLL_GAP 100UL // Small pause between completed nodes.
unsigned long lastPollAction = 0;

#define DUPLICATE_HISTORY_SIZE 10
String recentPacketKeys[DUPLICATE_HISTORY_SIZE];
byte recentPacketIndex = 0;

// MULTIHOP
#define PACKET_SEQUENCE_FILE "/packet_sequence.txt"
#define MAX_HOPS 1
unsigned long packetSequence = 0;

void sendAlertAck(String animalID,String alertType,String originalAlertPacketID,int maxHop)
{
  packetSequence++;
  savePacketSequence();

  LoRa.beginPacket();

  LoRa.print(F("MH,"));
  LoRa.print(packetSequence); // ACK packet's own ID
  LoRa.print(',');
  LoRa.print(F("BASE"));
  LoRa.print(',');
  LoRa.print(animalID);
```

```
  LoRa.print(F(",0,"));
  LoRa.print(maxHop);
  LoRa.print(',');
  LoRa.print(F("ACK,"));
  LoRa.print(animalID);
  LoRa.print(F(",ALERT,"));
  LoRa.print(alertType);
  LoRa.print(',');
  LoRa.print(originalAlertPacketID); // Original alert packet ID

  LoRa.endPacket();

  LoRa.receive();
#if DEBUG
  Serial.println(F("MULTIHOP ALERT ACK SENT"));

  Serial.print(F("ACK Packet ID: "));
  Serial.println(packetSequence);

  Serial.print(F("Acknowledging Alert ID: "));
  Serial.println(originalAlertPacketID);
#endif
}

// WEBSOCKET EVENT
void webSocketEvent(uint8_t num,WStype_t type,uint8_t *payload,size_t length)
{
  // CLIENT CONNECTED
  if (type == WStype_CONNECTED)
  {
    IPAddress ip = webSocket.remoteIP(num);
#if debug
    Serial.println();
    Serial.println("WebSocket client connected");
    Serial.print("Client IP: ");
    Serial.println(ip);
#endif
    // Send initial status
    webSocket.sendTXT(num,"STATUS,CONNECTED");
    return;
  }

  // CLIENT DISCONNECTED
  if (type == WStype_DISCONNECTED)
```

```
  {
#if debug
    Serial.println();
    Serial.println("WebSocket client disconnected");
#endif
    return;
  }

  // TEXT MESSAGE
  if (type == WStype_TEXT)
  {
    String command = String((char *)payload);
    command.trim();
#if DEBUG
    Serial.println();
    Serial.println("WebSocket command:");
    Serial.println(command);
#endif

    // Ignore empty command
    if (command.length() == 0)
    {
      return;
    }

    // GET STORED OFFLINE EVENTS
    if (command == "GET_EVENTS")
    {
      sendStoredEvents(num);
      return;
    }

    // EVENT ACKNOWLEDGMENT
    // Flutter sends:
    // EVENT_ACK,3
    if (command.startsWith("EVENT_ACK,"))
    {
      String eventIDString = command.substring(String("EVENT_ACK,").length());
      eventIDString.trim();
      unsigned long eventID = eventIDString.toInt();

      if (eventID == 0)
      {
#if DEBUG
```

```
      Serial.println(F("Invalid EVENT_ACK."));
#endif
      webSocket.sendTXT(num, "EVENT_ACK_FAILED,INVALID_ID");
      return;
    }

    bool success = acknowledgeEvent(eventID);

    if (success)
    {
      String response = "EVENT_ACK_SUCCESS," + String(eventID);
      webSocket.sendTXT(num, response);
    }
    else
    {
      String response = "EVENT_ACK_FAILED," + String(eventID);
      webSocket.sendTXT(num, response);
    }
    return;
  }

  // POLL is handled separately from normal commands
  if (command.startsWith("POLL,"))
  {
    sendPoll(command);
  }
  else
  {
    sendLoRaCommand(command);  // LIVE / FENCE / other commands
  }

  // Tell Flutter command was sent
  String response = "COMMAND_SENT," + command;
  webSocket.sendTXT(num, response);
 }
}

// POLL,A01
// POLL,A02
// POLL does NOT use the normal ACK/retry system.
// SEND POLL TO ANIMAL
void sendPoll(String command)
{
 command.trim();
```

```
String animalID = getAnimalID(command);

if (animalID.length() == 0)
{
  return;
}

int animalIndex = getAnimalIndex(animalID);

if (animalIndex < 0)
{
  return;
}

if (pollPending)
{
  return;
}

pollAnimalIndex = animalIndex;

// Create unique multihop packet ID
packetSequence++;
savePacketSequence();

LoRa.beginPacket();

LoRa.print(F("MH,"));
LoRa.print(packetSequence);
LoRa.print(',');
LoRa.print(F("BASE"));
LoRa.print(',');
LoRa.print(animalID);
LoRa.print(F(",0,"));
LoRa.print(MAX_HOPS);
LoRa.print(',');
LoRa.print(command);

LoRa.endPacket();

LoRa.receive();

pollPending = true;
```

```
  pollSendTime = millis();
}

// SEND COMMAND TO ANIMAL NODE
void sendLoRaCommand(String command)
{
  command.trim();

  if (command.length() == 0)
  {
    return;
  }

  // Extract target animal
  String animalID = getAnimalID(command);

  if (animalID.length() == 0)
  {
#if DEBUG
    Serial.println("Invalid command - no animal ID.");
#endif
    return;
  }

  // Cancel active automatic POLL
  if (pollPending)
  {
#if DEBUG
    Serial.println();
    Serial.println("Cancelling active POLL.");
    Serial.print("Previous animal: ");

    if (pollAnimalIndex >= 0 && pollAnimalIndex < MAX_ANIMAL_NODES)
    {
      Serial.println(animalIDs[pollAnimalIndex]);
    }
    else
    {
      Serial.println("UNKNOWN");
    }
#endif

    pollPending = false;
    pollAnimalIndex = -1;
```

```
    pollIndex++;     // Move scheduler to next animal

    if (pollIndex >= MAX_ANIMAL_NODES)
    {
      pollIndex = 0;
    }

    lastPollAction = millis();
  }

  // Add command to queue
  if (!queueCommand(command))
  {
    return;
  }

  // If another command is already active, leave this one in the queue.
  if (commandPending)
  {
#if DEBUG
    Serial.println("Command already active.");
    Serial.println("Waiting for previous command to finish.");
#endif
    return;
  }

  // Start next queued command
  startNextQueuedCommand();
}

// CHECK ACK PACKET
void processAck(String packet)
{
  // Expected formats:
  //
  // ACK,A02,LIVE,OFF,commandPacketID
  // ACK,A02,LIVE,ON,commandPacketID
  // ACK,A02,FENCE,OK,commandPacketID

  if (!packet.startsWith("ACK,"))
  {
    return;
  }
```

```
  // Parse ACK fields

  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);
  int comma4 = packet.indexOf(',', comma3 + 1);

  if (comma1 < 0 ||
      comma2 < 0 ||
      comma3 < 0 ||
      comma4 < 0)
  {
    return;
  }

  String animalID =
    packet.substring(comma1 + 1, comma2);

  String command =
    packet.substring(comma2 + 1, comma3);

  String value =
    packet.substring(comma3 + 1, comma4);

  String commandPacketIDText =
    packet.substring(comma4 + 1);

  animalID.trim();
  command.trim();
  value.trim();
  commandPacketIDText.trim();

  unsigned long commandPacketID =
    commandPacketIDText.toInt();


  // Is there a command waiting?

  if (!commandPending)
  {
#if DEBUG
    Serial.println("ACK received, but no command is pending.");
#endif
    return;
```

```
  }


  // Check animal

  if (animalID != pendingAnimalID)
  {
#if DEBUG
    Serial.println("ACK from another animal.");
#endif
    return;
  }


  // Extract expected command information

  bool validACK = false;


  // ========================
  // LIVE COMMAND
  // ========================

  if (pendingCommand.startsWith("LIVE,"))
  {
    // Example pending command:
    //
    // LIVE,A01,ON
    //
    // Expected ACK:
    //
    // ACK,A01,LIVE,ON,181

    String expectedAction =
      pendingCommand.substring(
        pendingCommand.lastIndexOf(',') + 1
      );

    expectedAction.trim();

    if (command == "LIVE" &&
        value == expectedAction)
    {
      validACK = true;
```

```
  }
 }


 // =========================
 // FENCE COMMAND
 // =========================

 else if (pendingCommand.startsWith("FENCE,"))
 {
  if (command == "FENCE" &&
      value == "OK")
  {
   validACK = true;
  }
 }


 // ACK does not match command

 if (!validACK)
 {
#if DEBUG
   Serial.println("ACK does not match pending command.");
#endif
   return;
 }


 // =========================
 // SUCCESS
 // =========================

 commandPending = false;


 // Update LIVE state

 if (pendingCommand.startsWith("LIVE,"))
 {
  int animalIndex =
   getAnimalIndex(animalID);

  if (animalIndex >= 0)
```

```
    {
      String action =
        pendingCommand.substring(
          pendingCommand.lastIndexOf(',') + 1
        );

      action.trim();

      if (action == "ON")
      {
        animalLive[animalIndex] = true;
      }
      else if (action == "OFF")
      {
        animalLive[animalIndex] = false;
      }
    }
  }


#if DEBUG

  Serial.println();
  Serial.println("================================");
  Serial.println("COMMAND ACKNOWLEDGED");

  Serial.print("Animal: ");
  Serial.println(animalID);

  Serial.print("Command: ");
  Serial.println(pendingCommand);

  Serial.print("Command Packet ID: ");
  Serial.println(commandPacketID);

  Serial.print("Attempts: ");
  Serial.println(commandRetryCount);

  Serial.println("================================");

#endif


  // Tell Flutter
```

```
  String response =
    "COMMAND_SUCCESS," + pendingCommand;

  webSocket.broadcastTXT(response);


  // Start next queued command

  startNextQueuedCommand();
}

// PROCESS GEOFENCE ALERT
// Expected:
// ALERT,A02,EXIT,27.68708,85.29596,1331.7
// ALERT,A01,RETURN,27.68708,85.29596,1331.7
void processAlert(String packet)
{
  processAlert(packet,"","",0);
}

void processAlert(String packet,String packetID,String source,int maxHop)
{
  if (!packet.startsWith("ALERT,"))
  {
    return;
  }

  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);

  if (comma1 < 0 || comma2 < 0 ||comma3 < 0)
  {
#if DEBUG
    Serial.println(F("Invalid ALERT packet."));
#endif
    return;
  }

  String animalID = packet.substring(comma1 + 1, comma2);
  String alertType = packet.substring(comma2 + 1, comma3);

  animalID.trim();
```

```
  alertType.trim();

  String eventData = packet.substring(comma3 + 1);

  eventData.trim();

  if (alertType != "EXIT" && alertType != "RETURN")
  {
    Serial.println(F("Invalid ALERT type."));
    return;
  }

  // SAVE ONLY NEW EVENT
  bool alreadySaved = isAlertAlreadySaved(animalID,packetID);

  if (alreadySaved)
  {
#if DEBUG
    Serial.println();
    Serial.println(F("================================"));
    Serial.println(F("DUPLICATE ALERT - PERSISTENT"));
    Serial.print(F("Animal: "));
    Serial.println(animalID);
    Serial.print(F("Alert Packet ID: "));
    Serial.println(packetID);
    Serial.println(F("Event already exists."));
    Serial.println(F("Resending ACK..."));
    Serial.println(F("================================"));
#endif

    sendAlertAck(animalID,alertType,packetID,maxHop);
    return;
  }
  saveEvent(animalID,packetID,alertType,eventData);

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("GEOFENCE ALERT RECEIVED"));
  Serial.print(F("Animal: "));
  Serial.println(animalID);
  Serial.print(F("Type: "));
  Serial.println(alertType);
```

```
  Serial.print(F("Packet ID: "));
  Serial.println(packetID);

  Serial.println(F("Packet: "));
  Serial.println(packet);
  Serial.println(F("================================"));
#endif

  // SEND ACK USING SAME ALERT PACKET ID

  sendAlertAck(animalID,alertType,packetID,maxHop);

  // FORWARD TO FLUTTER
  webSocket.broadcastTXT(packet);
}

// PROCESS POLL DATA
// Expected:
// DATA,A01,3.82,27.68708,85.29596,1331.7,5,1.67,0
// DATA itself is the response to POLL.
// PROCESS POLL DATA
void processPollData(String packet)
{
  if (!packet.startsWith("DATA,"))
  {
    return;
  }

  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);

  if (comma1 < 0 || comma2 < 0)
  {
#if DEBUG
    Serial.println("Invalid DATA packet.");
#endif
    return;
  }

  String animalID = packet.substring(comma1 + 1,comma2);
  animalID.trim();
#if DEBUG
  Serial.println();
  Serial.println("================================");
```

```
  Serial.println("POLL DATA RECEIVED");
  Serial.print("Animal: ");
  Serial.println(animalID);
  Serial.println(packet);
  Serial.println("================================");
#endif

  // Was this the animal we were polling?
  if (!pollPending)
  {
#if DEBUG
    Serial.println("DATA received without active POLL.");
#endif
    return;
  }

  if (pollAnimalIndex < 0 || pollAnimalIndex >= MAX_ANIMAL_NODES)
  {
#if DEBUG
    Serial.println("Invalid poll animal index.");
#endif
    return;
  }

  String expectedAnimal = animalIDs[pollAnimalIndex];

  if (animalID != expectedAnimal)
  {
#if DEBUG
    Serial.println("DATA from unexpected animal.");
#endif
    return;
  }

  // POLL SUCCESS
  pollPending = false;
  pollAnimalIndex = -1;

#if DEBUG
  Serial.println("POLL SUCCESS");
#endif
  webSocket.broadcastTXT(packet);   // Send telemetry to Flutter
  pollIndex++;  // MOVE TO NEXT ANIMAL
```

```
  if (pollIndex >= MAX_ANIMAL_NODES)
  {
    pollIndex = 0;
  }

#if DEBUG
  Serial.print("Next pollIndex: ");
  Serial.println(pollIndex);
  Serial.print("Next animal: ");
  Serial.println(animalIDs[pollIndex]);
#endif
  lastPollAction = millis();
}

// PROCESS MULTIHOP PACKET
// Format:
// MH,packetID,source,destination,hop,maxHop,payload
void processMultiHopPacket(String packet)
{
  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);
  int comma4 = packet.indexOf(',', comma3 + 1);
  int comma5 = packet.indexOf(',', comma4 + 1);
  int comma6 = packet.indexOf(',', comma5 + 1);

  if (comma1 < 0 || comma2 < 0 || comma3 < 0 || comma4 < 0 || comma5 < 0 || comma6 < 0)
  {
#if DEBUG
    Serial.println(F("Invalid multihop packet."));
#endif
    return;
  }

  // PARSE HEADER
  String packetID = packet.substring(comma1 + 1, comma2);
  String source = packet.substring(comma2 + 1, comma3);
  String destination = packet.substring(comma3 + 1, comma4);
  String hopText = packet.substring(comma4 + 1, comma5);
  String maxHopText = packet.substring(comma5 + 1, comma6);
  String payload = packet.substring(comma6 + 1);

  packetID.trim();
  source.trim();
```

```
  destination.trim();

  int hop = hopText.toInt();
  int maxHop = maxHopText.toInt();

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("MULTIHOP PACKET RECEIVED"));

  Serial.print(F("Source: "));
  Serial.println(source);

  Serial.print(F("Destination: "));
  Serial.println(destination);

  Serial.print(F("Packet ID: "));
  Serial.println(packetID);

  Serial.print(F("Hop: "));
  Serial.print(hop);
  Serial.print('/');
  Serial.println(maxHop);

  Serial.print(F("Payload: "));
  Serial.println(payload);

  Serial.println(F("================================"));
#endif

  // DUPLICATE DETECTION
  String packetKey = source + "," + packetID;

  bool duplicate = isDuplicatePacket(packetKey);

  if (!duplicate)
  {
    rememberPacket(packetKey);
  }

  // PACKET REACHED BASE
  if (destination == "BASE")
  {
    if (payload.startsWith("ACK,"))
```

```
    {
      processAck(payload);
    }
    else if (payload.startsWith("DATA,"))
    {
      if (!duplicate)
      {
        processPollData(payload);
      }
    #if DEBUG
      else
      {
        Serial.println(F("Duplicate DATA ignored."));
      }
    #endif
    }
    else if (payload.startsWith("ALERT,"))
    {
      if (duplicate)
      {
#if DEBUG
        Serial.println(F("Duplicate ALERT received - resending ACK"));
#endif
        int comma1 = payload.indexOf(',');
        int comma2 = payload.indexOf(',', comma1 + 1);
        int comma3 = payload.indexOf(',', comma2 + 1);

        if (comma1 >= 0 && comma2 >= 0 && comma3 >= 0)
        {
          String animalID = payload.substring(comma1 + 1,comma2);
          String alertType = payload.substring(comma2 + 1, comma3);

          animalID.trim();
          alertType.trim();

          if (alertType == "EXIT" || alertType == "RETURN")
          {
            sendAlertAck(animalID,alertType,packetID,maxHop);
          }
        }
      }
      else
      {
        processAlert(payload,packetID,source,maxHop);
```

```
      }
    }
    else
    {
#if DEBUG
      Serial.println(F("Unknown multihop payload."));
#endif
    }

    // IMPORTANT:
    // Destination reached.
    // Do NOT forward.
    return;
  }

  // BASE CAN ALSO ACT AS RELAY
  if (hop < maxHop)
  {
    hop++;
    LoRa.beginPacket();

    LoRa.print(F("MH,"));
    LoRa.print(packetID);
    LoRa.print(',');
    LoRa.print(source);
    LoRa.print(',');
    LoRa.print(destination);
    LoRa.print(',');
    LoRa.print(hop);
    LoRa.print(',');
    LoRa.print(maxHop);
    LoRa.print(',');
    LoRa.print(payload);

    LoRa.endPacket();

    LoRa.receive();
  }
}
// RECEIVE LORA PACKET
void receiveLoRaPacket()
{
  int packetSize = LoRa.parsePacket();
```

```
  if (!packetSize)
  {
    return;
  }

  String packet = "";

  while (LoRa.available())
  {
    packet += (char)LoRa.read();
  }

  packet.trim();
#if DEBUG
  Serial.println();
  Serial.println("================================");
  Serial.println("LoRa PACKET RECEIVED");
  Serial.println(packet);
  Serial.print("RSSI: ");
  Serial.print(LoRa.packetRssi());
  Serial.println(" dBm");
  Serial.print("SNR : ");
  Serial.print(LoRa.packetSnr());
  Serial.println(" dB");
  Serial.println("================================");
#endif

  // MULTIHOP PACKET
  if (packet.startsWith("MH,"))
  {
    processMultiHopPacket(packet);
  }
  else if (packet.startsWith("ACK,"))
  {
    processAck(packet);
  }
  else if (packet.startsWith("DATA,"))
  {
    processPollData(packet);
  }
  else if (packet.startsWith("ALERT,"))
  {
#if DEBUG
      Serial.println(F("Raw ALERT packet ignored - MH header required."));
```

```
#endif
  }
}

// AUTOMATIC POLL SCHEDULER
void updatePollScheduler()
{
  // Don't poll while a normal command is waiting for ACK
  if (commandPending || commandQueueCount > 0)
  {
    return;
  }

  // If waiting for DATA, check timeout
  if (pollPending)
  {
    if (millis() - pollSendTime >= POLL_TIMEOUT)
    {
#if DEBUG
      Serial.println();
      Serial.println("================================");
      Serial.println("POLL TIMEOUT");
      Serial.print("Animal: ");


      if (pollAnimalIndex >= 0 && pollAnimalIndex < MAX_ANIMAL_NODES)
      {
        Serial.println(animalIDs[pollAnimalIndex]);
      }
      else
      {
        Serial.println("UNKNOWN");
      }

      Serial.println("Moving to next animal.");
      Serial.println("================================");
#endif

      pollPending = false;
      pollAnimalIndex = -1;
      pollIndex++;

      if (pollIndex >= MAX_ANIMAL_NODES)
      {
```

```
      pollIndex = 0;
    }

    lastPollAction = millis();
  }

  return;
 }

 // Small gap between nodes
 //if (millis() - lastPollAction < POLL_GAP)
 //{
  //return;
 //}

 // Find next LIVE animal
 for (byte checked = 0; checked < MAX_ANIMAL_NODES;checked++)
 {
  if (animalLive[pollIndex])
  {
#if DEBUG
    Serial.println();
    Serial.println("---------- POLL SCHEDULER DEBUG ----------");
    Serial.print("pollIndex: ");
    Serial.println(pollIndex);
    Serial.print("Selected animal: ");
    Serial.println(animalIDs[pollIndex]);
    Serial.print("Live states: ");

    for (byte i = 0; i < MAX_ANIMAL_NODES; i++)
    {
      Serial.print(animalIDs[i]);
      Serial.print("=");

      if (animalLive[i]) Serial.print("ON");
      else
        Serial.print("OFF");

      Serial.print(" ");
    }

    Serial.println();
    Serial.println("-----------------------------------------");
#endif
```

```
      String command = "POLL," + animalIDs[pollIndex];
      sendPoll(command);
      return;
    }

    // Current animal is OFF
    pollIndex++;

    if (pollIndex >= MAX_ANIMAL_NODES)
    {
      pollIndex = 0;
    }
  }

  // No live animals
  lastPollAction = millis();
}

// HANDLE COMMAND RETRY
void updateCommandRetry()
{
  if (!commandPending)
  {
    return;
  }

  // Wait for ACK timeout
  if (millis() - lastCommandSend < COMMAND_RETRY_INTERVAL)
  {
    return;
  }

  // If maximum attempts have already been sent,
  // now declare failure.
  if (commandRetryCount >= COMMAND_MAX_RETRIES)
  {
    commandPending = false;
#if DEBUG
    Serial.println();
    Serial.println("================================");
    Serial.println("COMMAND FAILED");
    Serial.print("Animal: ");
    Serial.println(pendingAnimalID);
```

```
    Serial.print("Command: ");
    Serial.println(pendingCommand);
    Serial.println("No ACK received.");
    Serial.println("================================");
#endif

    String response = "COMMAND_FAILED," + pendingCommand;
    webSocket.broadcastTXT(response);
    startNextQueuedCommand(); // Try next queued command
    return;
  }

  // Retry
  commandRetryCount++;
#if DEBUG
  Serial.println();
  Serial.println("================================");
  Serial.println("RETRYING COMMAND");
  Serial.print("Attempt: ");
  Serial.print(commandRetryCount);
  Serial.print("/");
  Serial.println(COMMAND_MAX_RETRIES);
  Serial.print("Command: ");
  Serial.println(pendingCommand);
  Serial.println("================================");
#endif

  // Retry the same command with a new multihop packet ID
  packetSequence++;
  savePacketSequence();

  LoRa.beginPacket();

  LoRa.print(F("MH,"));
  LoRa.print(packetSequence);
  LoRa.print(',');
  LoRa.print(F("BASE"));
  LoRa.print(',');
  LoRa.print(pendingAnimalID);
  LoRa.print(F(",0,"));
  LoRa.print(MAX_HOPS);
  LoRa.print(',');
  LoRa.print(pendingCommand);
```

```
  LoRa.endPacket();
  LoRa.receive();

  lastCommandSend = millis();
}

// SETUP
void setup()
{
  // SERIAL
  Serial.begin(9600);
  delay(1000);
#if DEBUG
  Serial.println();
  Serial.println("================================");
  Serial.println("LIVESTOCK BASE STATION");
  Serial.println("ESP8266 + LoRa + WebSocket");
  Serial.println("================================");

  // LITTLEFS
  Serial.println();
  Serial.println(F("Initializing LittleFS..."));
#endif

  if (!LittleFS.begin())
  {
#if DEBUG
    Serial.println(F("LittleFS mount FAILED!"));
#endif
  }
  else
  {
#if DEBUG
    Serial.println(F("LittleFS ready."));
    Serial.println();
    Serial.println(F("================================"));
    Serial.println(F("PACKET ID LOADED"));
    Serial.print(F("Current Packet ID: "));
    Serial.println(packetSequence);
    Serial.print(F("Next Packet ID: "));
    Serial.println((uint16_t)(packetSequence + 1));
    Serial.println(F("================================"));
#endif
```

```
    //resetStoredData();
    loadPacketSequence();
    loadNextEventID();     // Load next event ID from stored events
  }

  WiFi.mode(WIFI_AP);
  WiFi.softAP(WIFI_SSID,WIFI_PASSWORD);
#if DEBUG
  // WIFI ACCESS POINT
  Serial.println();
  Serial.println("Starting WiFi...");
  Serial.println("WiFi AP started.");
  Serial.print("SSID: ");
  Serial.println(WIFI_SSID);
  Serial.print("IP address: ");
  Serial.println(WiFi.softAPIP());

  // LoRa
  Serial.println();
  Serial.println("Initializing LoRa...");
#endif

  SPI.begin();
  LoRa.setPins(LORA_SS,LORA_RST,LORA_DIO0);

  if (!LoRa.begin(LORA_FREQUENCY))
  {
#if DEBUG
    Serial.println("LoRa init FAILED!");
#endif
    while (true)
    {
      delay(1000);
    }
  }

  // LONG RANGE SETTINGS
  LoRa.setFrequency(LORA_FREQUENCY);
  LoRa.setTxPower(18);
  LoRa.setSpreadingFactor(12);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(8);
  LoRa.setPreambleLength(12);
```

```
#if DEBUG
  Serial.println(F("LoRa OK"));
  Serial.println(F("TX Power: 18 dBm"));
  Serial.println(F("SF: 12"));
  Serial.println(F("BW: 125 kHz"));
  Serial.println(F("CR: 4/8"));

  // WEBSOCKET
  Serial.println();
  Serial.println("Starting WebSocket server...");
#endif

  webSocket.begin();
  webSocket.onEvent(webSocketEvent);

#if DEBUG
  Serial.print("WebSocket server: ws://");
  Serial.print(WiFi.softAPIP());
  Serial.println(":81");

  // READY
  Serial.println();
  Serial.println("================================");
  Serial.println("BASE STATION READY");
  Serial.println("================================");
  Serial.println();
  Serial.println("Connect Flutter to WiFi:");
  Serial.println("BaseStation");
  Serial.println();
  Serial.println("WebSocket:");
  Serial.print("ws://");
  Serial.print( WiFi.softAPIP());
  Serial.println(":81");
  Serial.println();
#endif
}

void loop()
{

  webSocket.loop();

  receiveLoRaPacket();
```

```
  updateCommandRetry();

  updatePollScheduler();
}

// BASE STATION BATTERY
void batteryStatus()
{
  int adcValue = analogRead(BATTERY_PIN);
  float voltageA0 = adcValue * (3.2 / 1023.0);
  float batteryVoltage = voltageA0 * ((R1 + R2) / R2);
#if DEBUG
  Serial.print("Base battery: ");
  Serial.print(batteryVoltage,2);
  Serial.println(" V");
#endif
}

void resetStoredData()
{
  // RESET MULTIHOP PACKET ID
  if (LittleFS.exists(PACKET_SEQUENCE_FILE))
  {
    LittleFS.remove(PACKET_SEQUENCE_FILE);
  }

  packetSequence = 0;

  // RESET OFFLINE EVENTS
  if (LittleFS.exists(EVENTS_FILE))
  {
    LittleFS.remove(EVENTS_FILE);
  }

  // Remove temporary event file too
  if (LittleFS.exists(EVENTS_TEMP_FILE))
  {
    LittleFS.remove(EVENTS_TEMP_FILE);
  }

  // Reset RAM event counter
  nextEventID = 1;

#if DEBUG
```

```
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("STORED DATA RESET"));
  Serial.println(F("Packet ID reset to: 0"));
  Serial.println(F("Event ID reset to: 1"));
  Serial.println(F("Events file deleted."));
  Serial.println(F("================================"));
#endif
}

void loadPacketSequence()
{
  if (!LittleFS.exists(PACKET_SEQUENCE_FILE))
  {
    packetSequence = 0;
    return;
  }

  File file = LittleFS.open(PACKET_SEQUENCE_FILE,"r");

  if (!file)
  {
    packetSequence = 0;
    return;
  }

  String value = file.readStringUntil('\n');
  file.close();
  value.trim();
  unsigned long loaded = value.toInt();

  // Invalid / empty value
  if (value.length() == 0)
  {
    packetSequence = 0;
    return;
  }

  packetSequence = (uint16_t)loaded;
}

void savePacketSequence()
{
  File file = LittleFS.open(PACKET_SEQUENCE_FILE,"w");
```

```
  if (!file)
  {
#if DEBUG
    Serial.println(F("ERROR: Cannot save packet sequence"));
#endif
    return;
  }

  file.println(packetSequence);
  file.close();
}

bool isDuplicatePacket(String packetKey)
{
  for (byte i = 0; i < DUPLICATE_HISTORY_SIZE; i++)
  {
    if (recentPacketKeys[i] == packetKey)
    {
      return true;
    }
  }

  return false;
}

void rememberPacket(String packetKey)
{
  recentPacketKeys[recentPacketIndex] = packetKey;

  recentPacketIndex++;

  if (recentPacketIndex >= DUPLICATE_HISTORY_SIZE)
  {
    recentPacketIndex = 0;
  }
}

// COMMAND QUEUE
bool isCommandQueueFull()
{
  return commandQueueCount >= COMMAND_QUEUE_SIZE;
}
```

```
// ADD COMMAND TO QUEUE
bool queueCommand(String command)
{
  command.trim();

  if (command.length() == 0)
  {
    return false;
  }

  if (isCommandQueueFull())
  {
#if DEBUG
    Serial.println();
    Serial.println("================================");
    Serial.println("COMMAND QUEUE FULL");
    Serial.println(command);
    Serial.println("================================");
#endif
    return false;
  }

  commandQueue[commandQueueTail] = command;
  commandQueueTail++;

  if (commandQueueTail >= COMMAND_QUEUE_SIZE)
  {
    commandQueueTail = 0;
  }

  commandQueueCount++;
#if DEBUG
  Serial.println();
  Serial.println("COMMAND QUEUED");
  Serial.print("Command: ");
  Serial.println(command);
  Serial.print("Queue count: ");
  Serial.println(commandQueueCount);
#endif
  return true;
}

// GET NEXT COMMAND FROM QUEUE
String dequeueCommand()
```

```
{
  if (commandQueueCount == 0)
  {
    return "";
  }

  String command = commandQueue[commandQueueHead];
  commandQueueHead++;

  if (commandQueueHead >= COMMAND_QUEUE_SIZE)
  {
    commandQueueHead = 0;
  }

  commandQueueCount--;
  return command;
}

// EXTRACT ANIMAL ID
// Examples:
// LIVE,A01,ON
//       └── A01
String getAnimalID(String command)
{
  int firstComma = command.indexOf(',');

  if (firstComma < 0)
  {
    return "";
  }

  int secondComma = command.indexOf(',', firstComma + 1);
  String animalID;

  if (secondComma < 0)
  {
    // Command has only two fields
    // Example: POLL,A01
    animalID = command.substring(firstComma + 1);
  }
  else
  {
    // Example: LIVE,A01,OFF
    //          FENCE,A01,4,...
```

```
    animalID = command.substring(firstComma + 1,secondComma);
  }

  animalID.trim();
  return animalID;
}

// FIND ANIMAL INDEX
int getAnimalIndex(String animalID)
{
  animalID.trim();

  for (byte i = 0; i < MAX_ANIMAL_NODES; i++)
  {
    if (animalIDs[i] == animalID)
    {
      return i;
    }
  }

  return -1;
}

// START NEXT QUEUED COMMAND
void startNextQueuedCommand()
{
  // Already processing a command
  if (commandPending)
  {
    return;
  }

  // Nothing waiting
  if (commandQueueCount == 0)
  {
    return;
  }

  String command = dequeueCommand();

  if (command.length() == 0)
  {
    return;
```

```
  }

  String animalID = getAnimalID(command);

  if (animalID.length() == 0)
  {
#if DEBUG
    Serial.println("Invalid queued command.");
#endif
    return;
  }

  // Save pending command
  pendingCommand = command;
  pendingAnimalID = animalID;
  commandPending = true;
  commandRetryCount = 1;

#if DEBUG
  Serial.println();
  Serial.println("================================");
  Serial.println("SENDING QUEUED COMMAND");
  Serial.print("Command: ");
  Serial.println(pendingCommand);
  Serial.print("Target animal: ");
  Serial.println(pendingAnimalID);
  Serial.print("Remaining queue: ");
  Serial.println(commandQueueCount);
  Serial.println("================================");
#endif

  // Create multihop packet ID
  packetSequence++;
  savePacketSequence();

  // Send command through multihop
  LoRa.beginPacket();

  LoRa.print(F("MH,"));
  LoRa.print(packetSequence);
  LoRa.print(',');
  LoRa.print(F("BASE"));
  LoRa.print(',');
  LoRa.print(pendingAnimalID);
```

```
  LoRa.print(F(",0,"));
  LoRa.print(MAX_HOPS);
  LoRa.print(',');
  LoRa.print(pendingCommand);

  LoRa.endPacket();
  LoRa.receive();

  lastCommandSend = millis();
#if DEBUG
  Serial.println("LoRa command sent.");
  Serial.println("Waiting for ACK...");
#endif
}

// SAVE EVENT TO LITTLEFS
// Stored format:
// EVENT_ID,ANIMAL_ID,TYPE,LAT,LON,ALT
// Example:
// 1,A02,EXIT,27.68708,85.29596,1331.7
void saveEvent(String animalID,String alertPacketID,String alertType,String eventData)
{
  File file = LittleFS.open(EVENTS_FILE, "a");

  if (!file)
  {
#if DEBUG
    Serial.println(F("Failed to open events file."));
#endif
    return;
  }

  String event = "";

  event += String(nextEventID);
  event += ",";
  event += animalID;
  event += ",";
  event += alertPacketID;
  event += ",";
  event += alertType;
  event += ",";
  event += eventData;
```

```
  file.println(event);
  file.close();

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("OFFLINE EVENT SAVED"));

  Serial.print(F("Event ID: "));
  Serial.println(nextEventID);

  Serial.print(F("Animal: "));
  Serial.println(animalID);

  Serial.print(F("Alert Packet ID: "));
  Serial.println(alertPacketID);

  Serial.print(F("Type: "));
  Serial.println(alertType);

  Serial.print(F("Event: "));
  Serial.println(event);

  Serial.println(F("================================"));
#endif

  nextEventID++;
}

// LOAD NEXT EVENT ID
// Reads existing events from LittleFS and finds the highest Event ID.
// Example file:
// 1,A01,EXIT,...
// 2,A02,RETURN,...
// 3,A03,EXIT,...
// After loading: nextEventID = 4
void loadNextEventID()
{
  File file = LittleFS.open(EVENTS_FILE, "r");

  // No events file yet
  if (!file)
  {
#if DEBUG
```

```
    Serial.println(F("No existing events file."));
#endif
    nextEventID = 1;
    return;
  }

  unsigned long highestID = 0;

  while (file.available())
  {
    String line = file.readStringUntil('\n');

    line.trim();

    if (line.length() == 0)
    {
      continue;
    }

    // First field before comma is Event ID
    int commaIndex = line.indexOf(',');

    if (commaIndex < 0)
    {
      continue;
    }

    String idString = line.substring(0, commaIndex);
    idString.trim();
    unsigned long eventID = idString.toInt();

    if (eventID > highestID)
    {
      highestID = eventID;
    }
  }

  file.close();

  nextEventID = highestID + 1;

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
```

```
  Serial.println(F("EVENT ID LOADED"));
  Serial.print(F("Highest Event ID: "));
  Serial.println(highestID);
  Serial.print(F("Next Event ID: "));
  Serial.println(nextEventID);
  Serial.println(F("================================"));
#endif
}

// SEND STORED EVENTS TO FLUTTER
// Flutter sends:
// GET_EVENTS
// BaseStation responds:
// EVENT,1,A02,EXIT,0,0,0
// EVENT,2,A02,RETURN,0,0,0
// EVENTS_END
void sendStoredEvents(uint8_t clientNum)
{
  File file = LittleFS.open(EVENTS_FILE, "r");

  // No events stored
  if (!file)
  {
#if DEBUG
    Serial.println(F("No stored events file."));
#endif
    webSocket.sendTXT(clientNum, "EVENTS_EMPTY");
    return;
  }

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("SENDING STORED EVENTS"));
  Serial.println(F("================================"));
#endif

  uint16_t eventCount = 0;

  while (file.available())
  {
    String event = file.readStringUntil('\n');
    event.trim();
```

```
    // Ignore empty lines
    if (event.length() == 0)
    {
      continue;
    }

    // Stored:
    // 1,A02,EXIT,0,0,0
    // Send to Flutter:
    // EVENT,1,A02,EXIT,0,0,0

    String message = "EVENT," + event;
    webSocket.sendTXT(clientNum, message);
#if DEBUG
    Serial.println(message);
#endif
    eventCount++;
    // Small gap between WebSocket messages
    delay(20);
  }

  file.close();
#if DEBUG
  Serial.println();
  Serial.print(F("Total events sent: "));
  Serial.println(eventCount);
#endif

  // Tell Flutter transmission is complete
  webSocket.sendTXT(clientNum, "EVENTS_END");
}

// ACKNOWLEDGE EVENT
// Flutter sends:
// EVENT_ACK,3
// This removes Event ID 3 from LittleFS.
// All other events remain.
bool acknowledgeEvent(unsigned long eventID)
{
  File sourceFile = LittleFS.open(EVENTS_FILE, "r");

  if (!sourceFile)
  {
#if DEBUG
```

```
    Serial.println(F("Events file not found."));
#endif
    return false;
  }

  // Remove old temporary file if it exists
  if (LittleFS.exists(EVENTS_TEMP_FILE))
  {
    LittleFS.remove(EVENTS_TEMP_FILE);
  }

  File tempFile = LittleFS.open(EVENTS_TEMP_FILE, "w");

  if (!tempFile)
  {
#if DEBUG
    Serial.println(F("Failed to create temporary events file."));
#endif
    sourceFile.close();
    return false;
  }

  bool eventFound = false;

  while (sourceFile.available())
  {
    String line = sourceFile.readStringUntil('\n');
    line.trim();

    if (line.length() == 0)
    {
      continue;
    }

    // Extract Event ID from beginning of line
    int commaIndex = line.indexOf(',');

    if (commaIndex < 0)
    {
      // Keep malformed lines rather than silently losing them
      tempFile.println(line);
      continue;
    }
```

```
    String idString = line.substring(0, commaIndex);
    idString.trim();

    unsigned long currentEventID = idString.toInt();

    // Skip the event being acknowledged
    if (currentEventID == eventID)
    {
      eventFound = true;
#if DEBUG
      Serial.print(F("Removing acknowledged event: "));
      Serial.println(eventID);
#endif
      continue;
    }

    // Keep all other events
    tempFile.println(line);
  }

  sourceFile.close();
  tempFile.close();

  // Event was not found, so don't replace original file
  if (!eventFound)
  {
    LittleFS.remove(EVENTS_TEMP_FILE);
#if DEBUG
    Serial.print(F("Event ID not found: "));
    Serial.println(eventID);
#endif
    return false;
  }

  // Remove original file
  if (!LittleFS.remove(EVENTS_FILE))
  {
#if DEBUG
    Serial.println(F("Failed to remove original events file."));
#endif
    return false;
  }

  // Rename temporary file
```

```
  if (!LittleFS.rename(EVENTS_TEMP_FILE, EVENTS_FILE))
  {
#if DEBUG
    Serial.println(F("Failed to replace events file."));
#endif
    return false;
  }

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("EVENT ACKNOWLEDGED"));
  Serial.print(F("Event ID: "));
  Serial.println(eventID);
  Serial.println(F("================================"));
#endif

  return true;
}

bool isAlertAlreadySaved(String animalID,String alertPacketID)
{
  File file = LittleFS.open(EVENTS_FILE, "r");

  if (!file)
  {
    return false;
  }

  while (file.available())
  {
    String line = file.readStringUntil('\n');

    line.trim();

    if (line.length() == 0)
    {
      continue;
    }

    // EVENT FORMAT:
    // EventID,AnimalID,AlertPacketID,AlertType,...

    int comma1 = line.indexOf(',');
```

```
    if (comma1 < 0)
      continue;

    int comma2 = line.indexOf(',', comma1 + 1);
    if (comma2 < 0)
      continue;

    int comma3 = line.indexOf(',', comma2 + 1);
    if (comma3 < 0)
      continue;

    String storedAnimal =
      line.substring(
        comma1 + 1,
        comma2
      );

    String storedPacketID =
      line.substring(
        comma2 + 1,
        comma3
      );

    storedAnimal.trim();
    storedPacketID.trim();

    if (
      storedAnimal == animalID &&
      storedPacketID == alertPacketID
    )
    {
      file.close();
      return true;
    }
  }

  file.close();

  return false;
}
```